

Presented By: Omar Ahmed

16/7/2026

Machine Learning Optimization

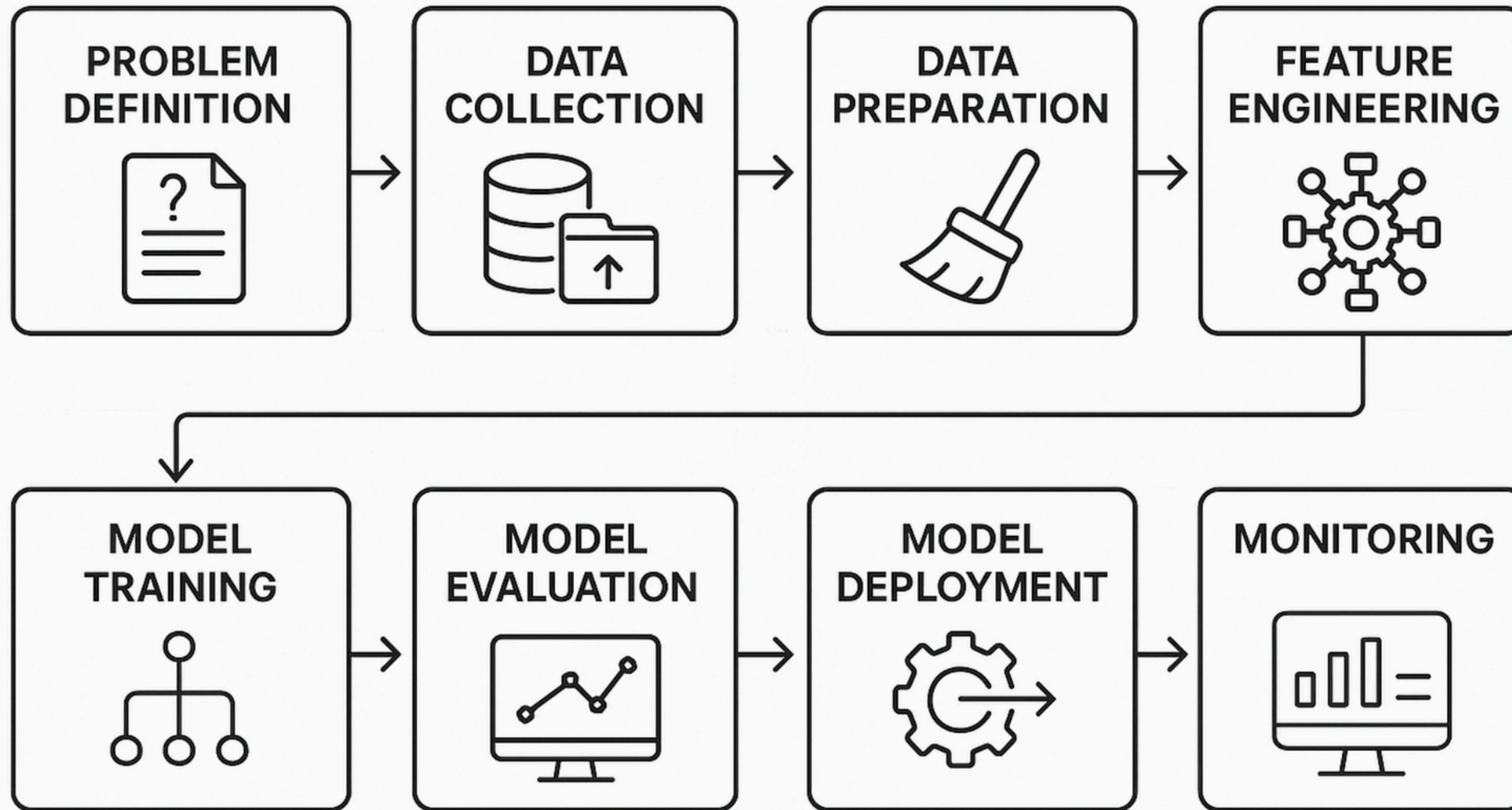
Build Better ML Models with practical
Explanations



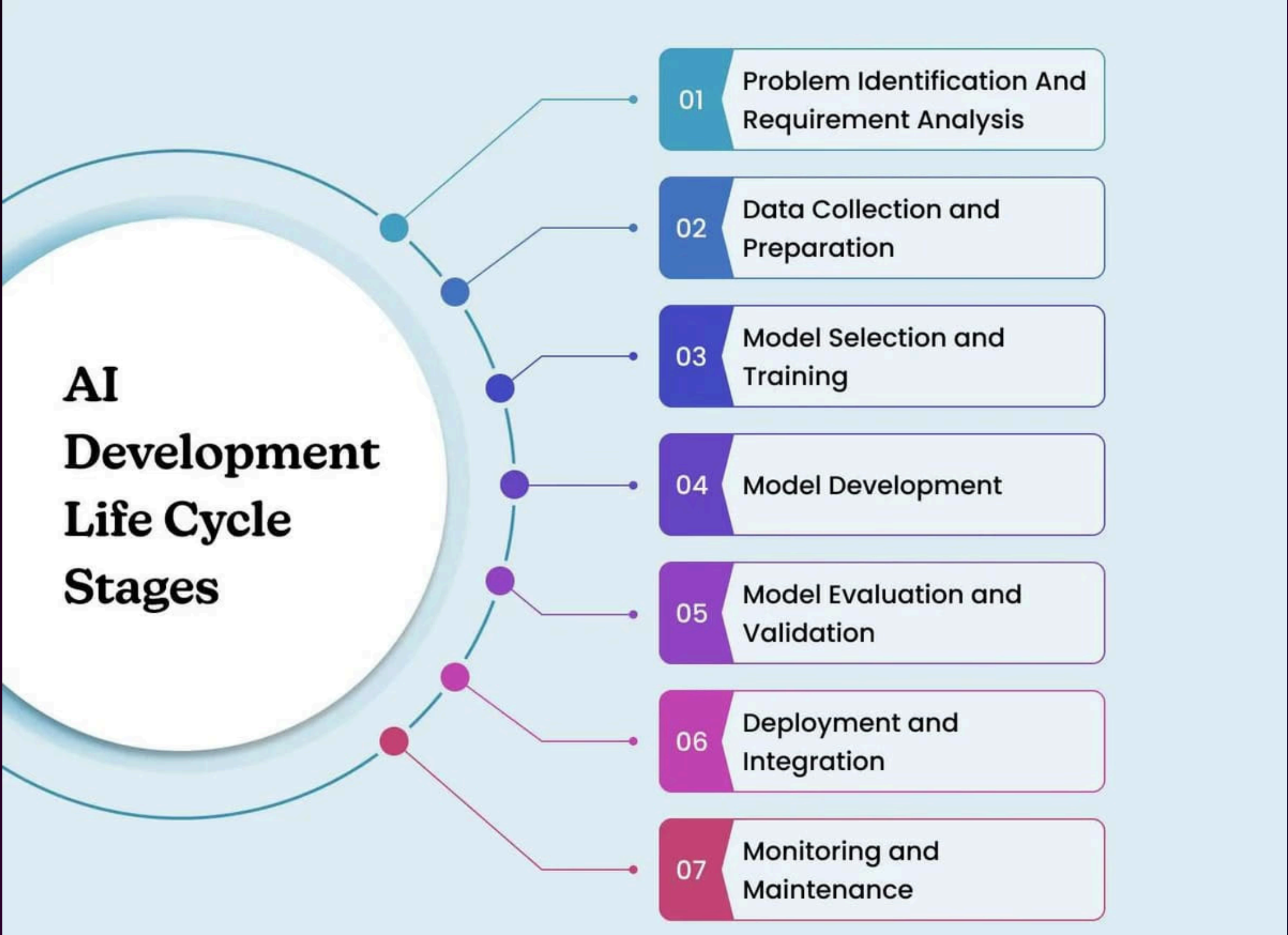
Topics
Life Cycle of AI
Regression test methods
Classification test methods
Hyperparameters tuning
Features extraction
Features Selection
Ensemble Learning

AI Life Cycle

AI LIFECYCLE



AI Development Life Cycle Stages



```
graph TD; 01[01 Problem Identification And Requirement Analysis] --> 02[02 Data Collection and Preparation]; 02 --> 03[03 Model Selection and Training]; 03 --> 04[04 Model Development]; 04 --> 05[05 Model Evaluation and Validation]; 05 --> 06[06 Deployment and Integration]; 06 --> 07[07 Monitoring and Maintenance]; 07 --> 01;
```

01

Problem Identification And
Requirement Analysis

02

Data Collection and
Preparation

03

Model Selection and
Training

04

Model Development

05

Model Evaluation and
Validation

06

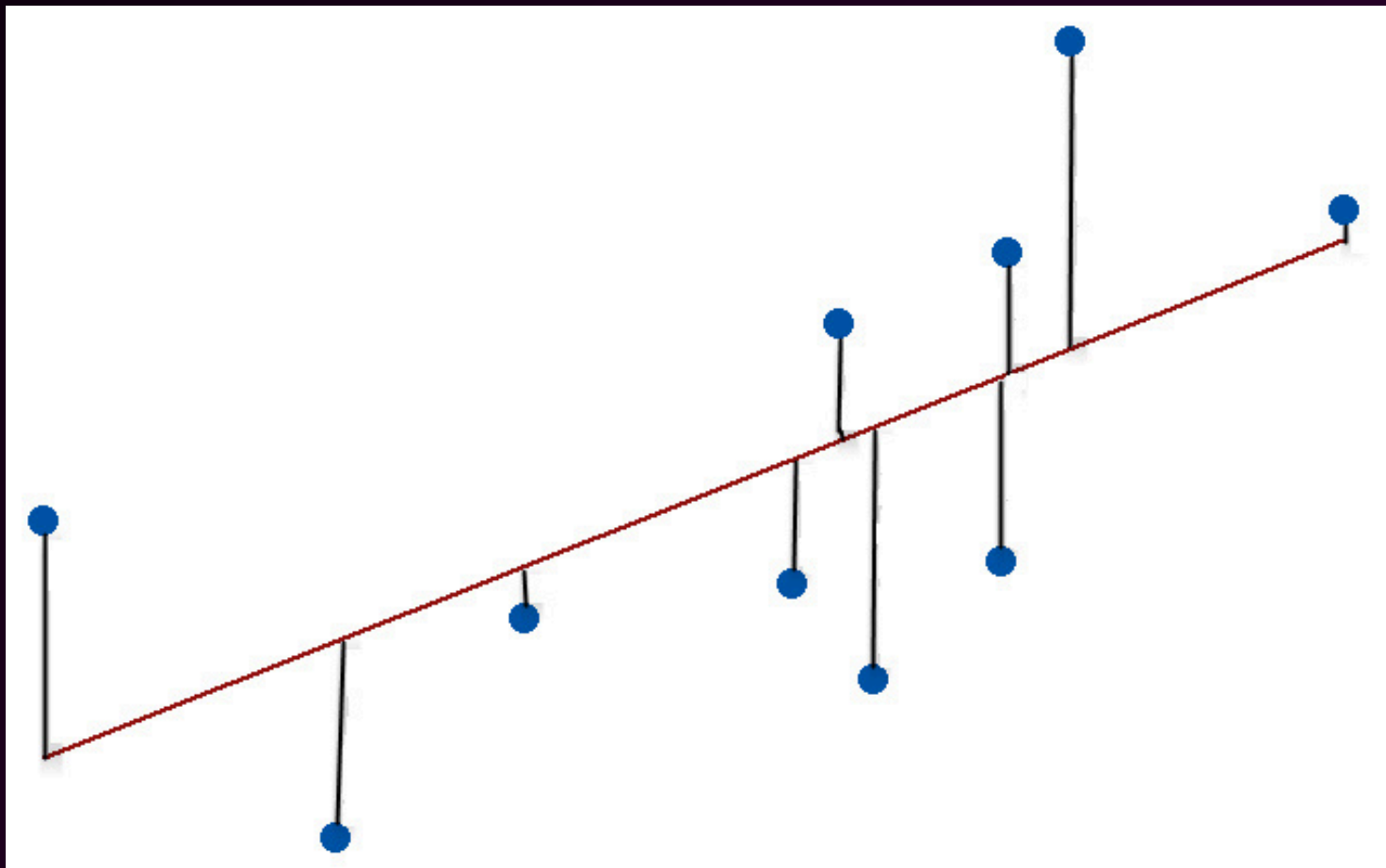
Deployment and
Integration

07

Monitoring and
Maintenance

Regression test methods

MSE (mean squared error)



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MSE measures the average squared difference between the predicted and actual values

pros :

- **penalized larger errors more heavily**
- **Differentiable, making it suitable for optimization algorithms like gradient Descent**

cons :

- **Sensitive to outliers, as large errors are amplified**

MAE (mean absolute error)

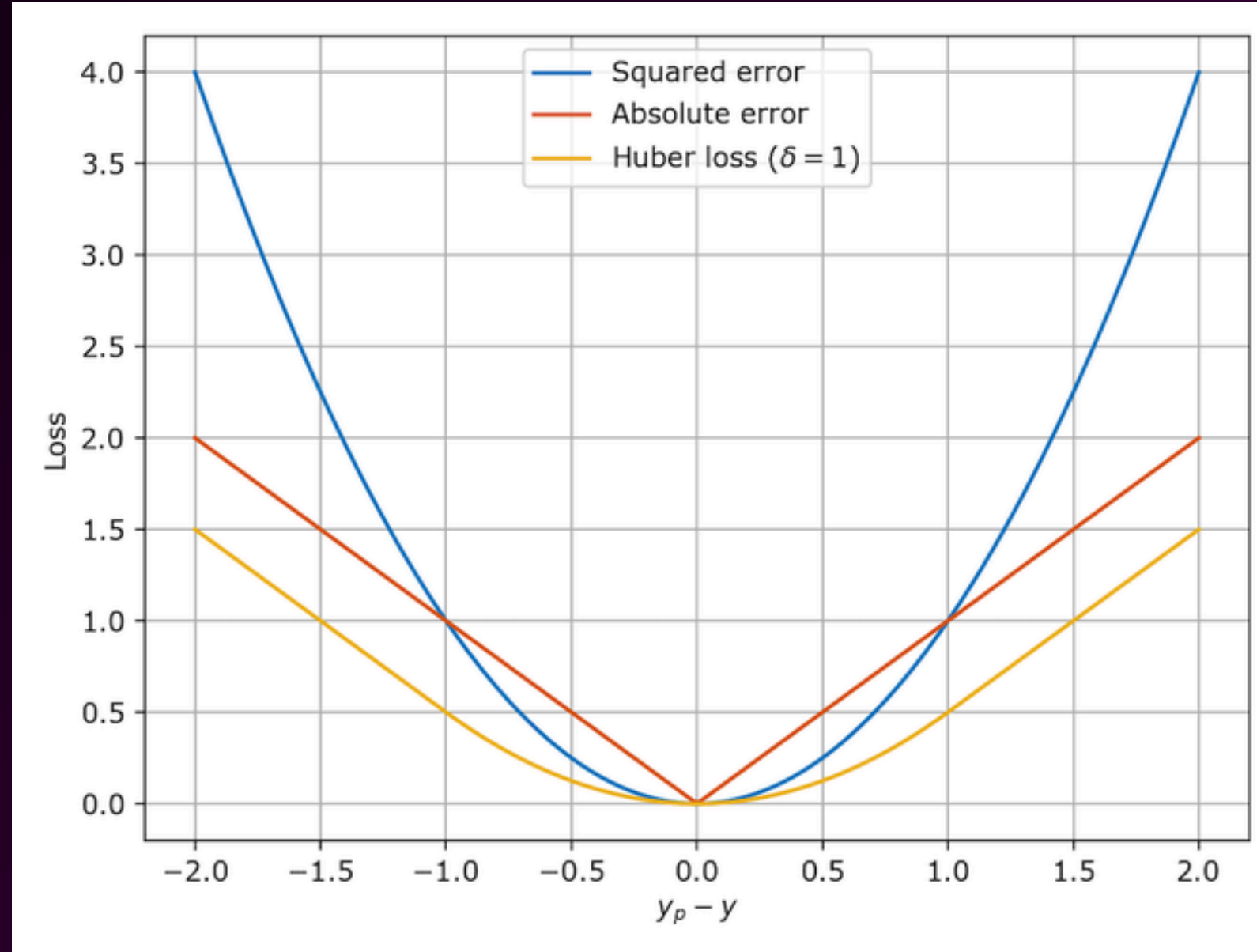
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

**less sensitive to outliers compared to MSE, but non-differentiable, which can make optimization challenging
used when robustness to outliers is needed**

RMSE (Root mean squared error)

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

Huber Loss



$$L_{\delta}(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta \\ \delta \cdot (|y - f(x)| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

R-Squared

$$R^2 = 1 - \frac{SS_{Regression}}{SS_{Total}}$$

Sum Squared Regression Error

Sum Squared Total Error

perfect if = 1
poor if = 0

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

R-Squared

- Not always inductor of Godness, so High R Squared doesn't mean model is Good, it may be influenced by Overfitting



```
from sklearn.metrics import (
    mean_squared_error,
    root_mean_squared_error,
    mean_absolute_error,
    r2_score)
from sklearn.linear_model import HuberRegressor

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

rmse = root_mean_squared_error(y_test, y_pred)

mae = mean_absolute_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print(f"MSE : {mse:.4f}")
print(f"RMSE : {rmse:.4f}")
print(f"MAE : {mae:.4f}")
print(f"R2 : {r2:.4f}")
```



```
from sklearn.linear_model import HuberRegressor

model = HuberRegressor()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MSE : {mse:.4f}")
print(f"RMSE : {rmse:.4f}")
print(f"MAE : {mae:.4f}")
print(f"R2 : {r2:.4f}")
```

Classification test methods

TP

model correctly predicts positive

Actual = 1
predicted = 1

TN

model correctly predicts Negative

Actual = 0
predicted = 0

FP

model predicts positive, but it is
Negative

Actual = 0
predicted = 1

FN

model predicts Negative, but it is
Positive

Actual = 1
predicted = 0

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

102	18
9	110

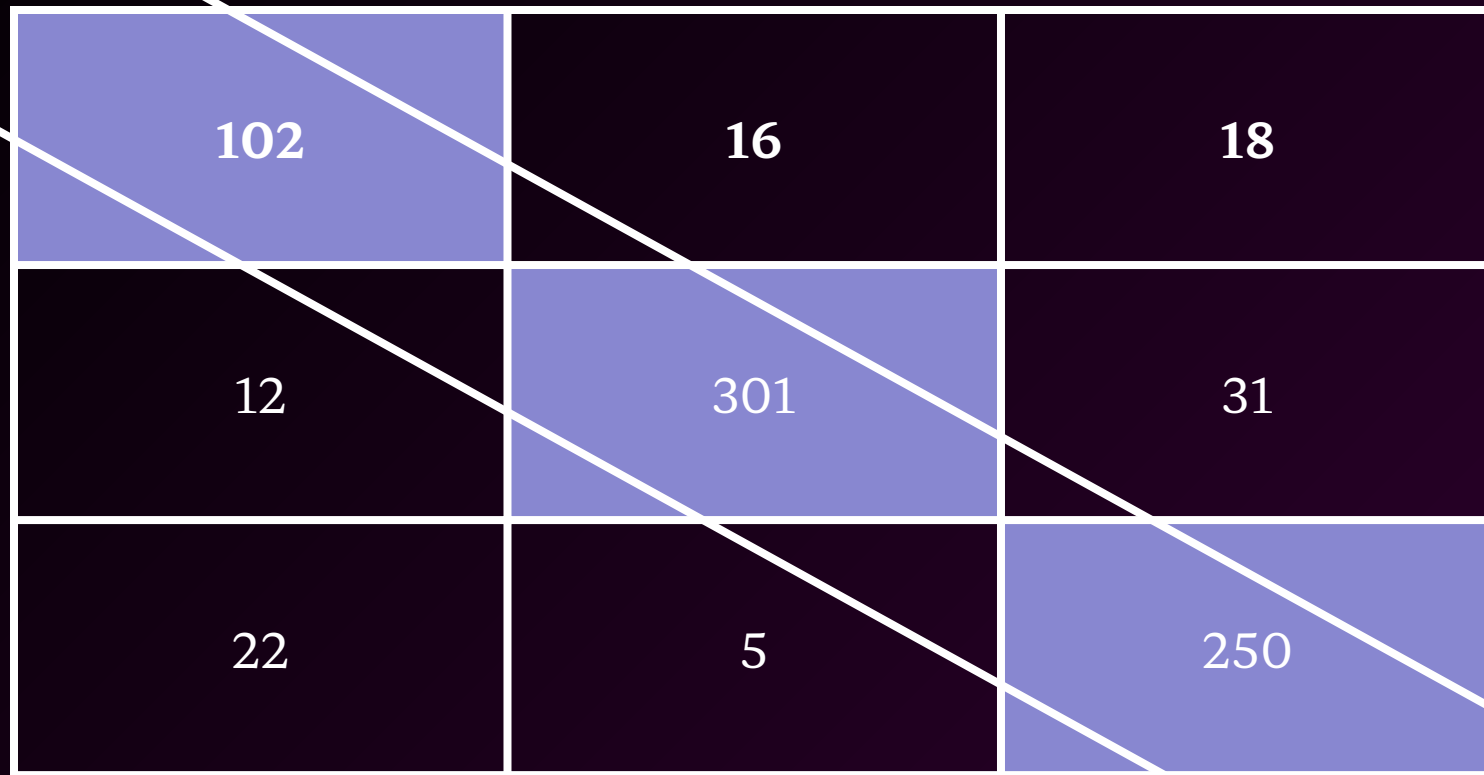
TP = 102

TN = 110

FN = 18

FP = 9

$$\text{ACC} = (102 + 110) / (102 + 110 + 18 + 9) = 88.702\%$$



102	16	18
12	301	31
22	5	250

$$\text{ACC} = (102 + 301 + 250) / (\text{All}) = 86.26\%$$

Precision

Precision is how many of positives predicted by your model were actually correct

$$\text{precision} = TP / (TP + FP)$$

30	2	3
4	25	1
2	3	35

$$\begin{aligned}\text{precision A} &= 30 / (30 + 6) \\ \text{precision B} &= 25 / (25 + 5) \\ \text{precision C} &= 35 / (35 + 4)\end{aligned}$$

$$\begin{aligned}\text{Macro precision} &= \text{Avg}(P1, P2, P3) \\ &= 0.854687\end{aligned}$$

Recall

Use when False Negatives are costly, Of all actual positive samples, how many did the model find?

$$\text{Recall} = TP / (TP + FN)$$

30	2	3
4	25	1
2	3	35

$$\begin{aligned}\text{Recall A} &= 30 / (30 + 5) \\ \text{Recall B} &= 25 / (25 + 5) \\ \text{Recall C} &= 35 / (35 + 5)\end{aligned}$$

$$\begin{aligned}\text{Macro Recall} &= \text{Avg}(P1, P2, P3) \\ &= 0.855158\end{aligned}$$

Precision answers: "When the model says an email is spam, how often is it correct?"

High precision means the model rarely labels a normal email as spam (few false positives).

Recall answers: "Of all the actual spam emails, how many did the model successfully find?"

High recall means the model misses very few spam emails (few false negatives).

F1 Score

$$\text{F1} = 2 \times (\text{precision} \times \text{Recall}) / (\text{precision} + \text{Recall})$$

Balances Precision and Recall.



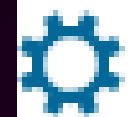
$$\text{F1 Score} = \frac{2}{\left(\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}} \right)}$$

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

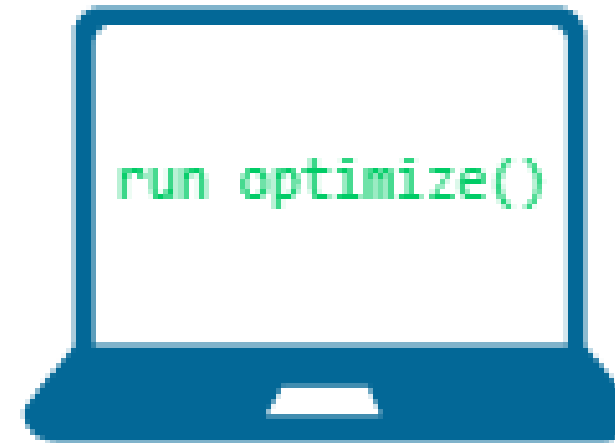
Hyperparameters Tuning



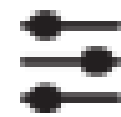
Hyperparameters



n_layers = 3
n_neurons = 512
learning_rate = 0.1



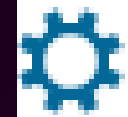
Parameters



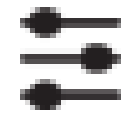
Weights
optimization



Score



n_layers = 3
n_neurons = 1024
learning_rate = 0.01

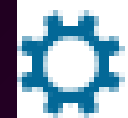


Weights
optimization

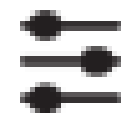


85%

80%



n_layers = 5
n_neurons = 256
learning_rate = 0.1



Weights
optimization



92%

Method	Idea	Advantages	Disadvantages
1. Grid Search	Try every possible combination of hyperparameters.	Finds the best combination within the defined grid.	Very slow for many hyperparameters.
2. Random Search	Randomly sample combinations from the search space.	Much faster than Grid Search and often finds similar or better results.	May miss the absolute best combination.
3. Bayesian Optimization	Uses previous results to intelligently choose the next hyperparameters to test.	Very efficient; requires fewer evaluations.	More complex to implement.
4. Evolutionary (Genetic) Algorithms	Mimics natural evolution using selection, crossover, and mutation to evolve better hyperparameters.	Good for large and complex search spaces.	Computationally expensive and slower to converge.
5. Hyperband / Successive Halving	Quickly discards poorly performing hyperparameter configurations and allocates more resources to promising ones.	Extremely efficient for expensive models like deep neural networks.	Can stop potentially good configurations too early if they start slowly.

Limitation of Grid Search and Random Search



blog.DailyDoseofDS.com

	Grid Search	Random Search	Bayesian Optimization		
Total Iterations	720	360	100	7x less iterations	✓
Total Run-time	242s	113s	48s	5x less run-time	✓
Best Trial Index	667	221	83	Optimal config. found earlier	✓
Best F1 score	0.93	0.93	0.93	Same score	✓

Grid Search

Grid Search is a hyperparameter tuning technique that finds the best combination of hyperparameters by testing every possible combination from a predefined set of values. Hyperparameters are settings chosen before training, such as `max_depth` for a Decision Tree or `n_estimators` for a Random Forest.

For example, suppose you want to tune:

- `max_depth = [3, 5, 7]`
- `min_samples_split = [2, 4]`

Grid Search trains and evaluates the model for all possible combinations:

- `(3, 2)`
- `(3, 4)`
- `(5, 2)`
- `(5, 4)`
- `(7, 2)`
- `(7, 4)`



```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
```

```
X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

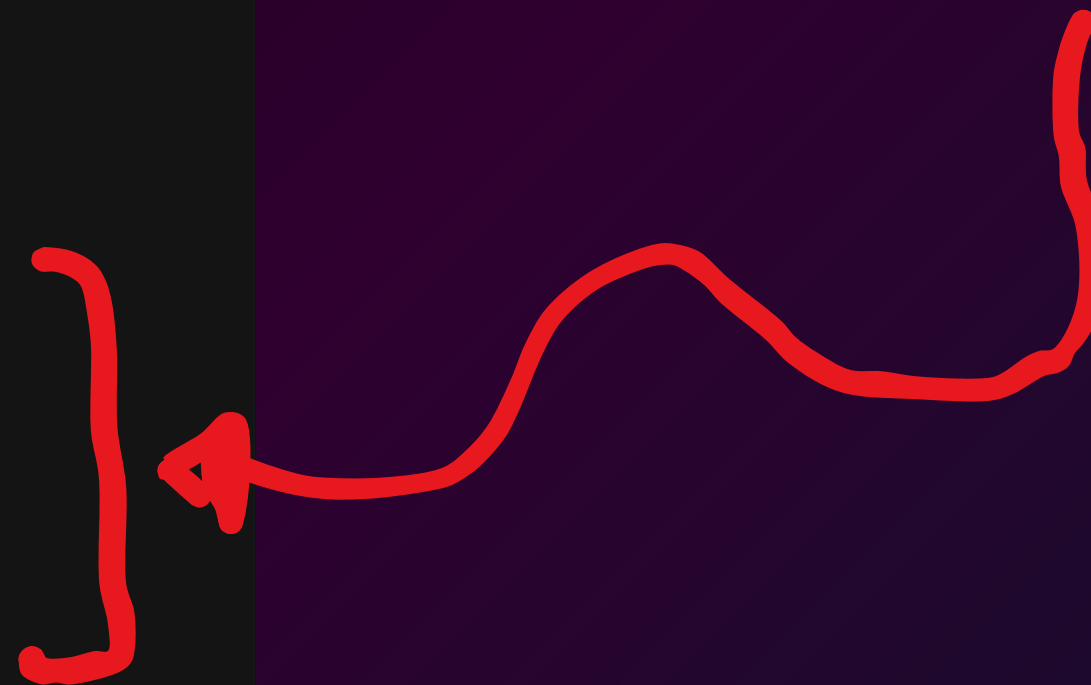
```
param_grid = {
    "max_depth": [3, 5, 7],
    "min_samples_split": [2, 4, 6]
}
```

```
grid = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring="accuracy"
)
```

```
grid.fit(X_train, y_train)
```

```
print(grid.best_params_)
print(grid.best_score_)
print(grid.score(X_test, y_test))
```

Grid Search model



Random

Search is a hyperparameter tuning technique that randomly selects combinations of hyperparameters from a predefined search space instead of trying every possible combination like Grid Search.

For example, suppose you have:

`max_depth = [3, 5, 7, 9, 11]`

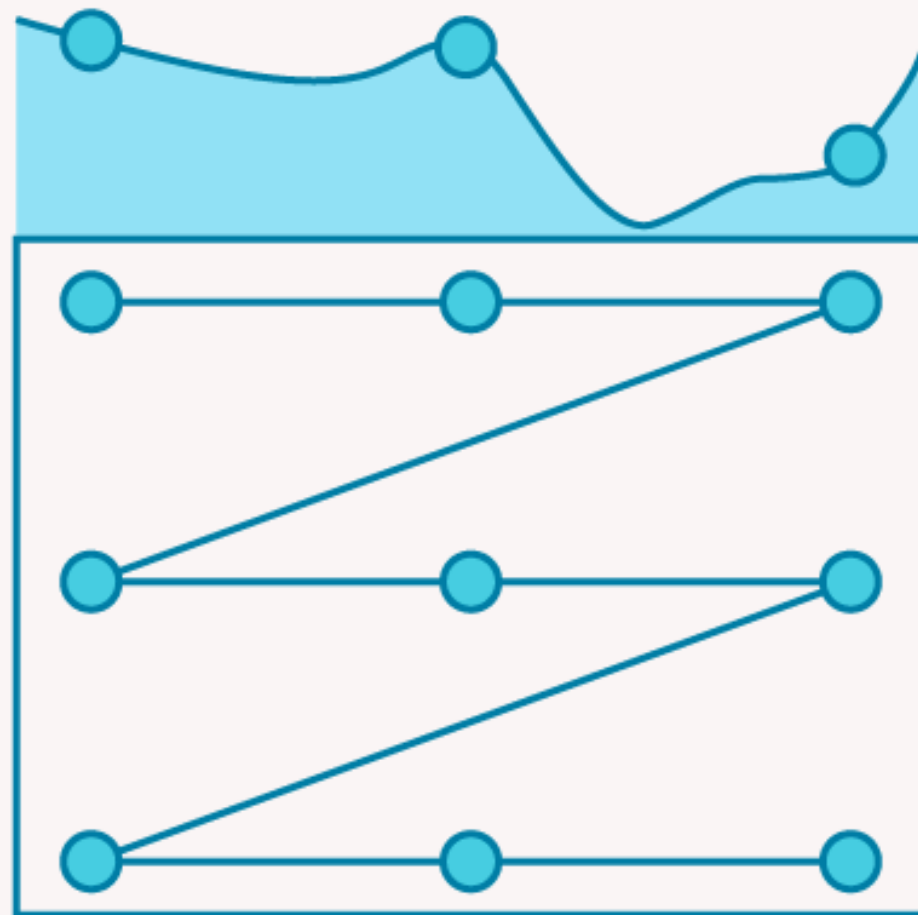
`min_samples_split = [2, 4, 6, 8]`

`criterion = ["gini", "entropy"]`

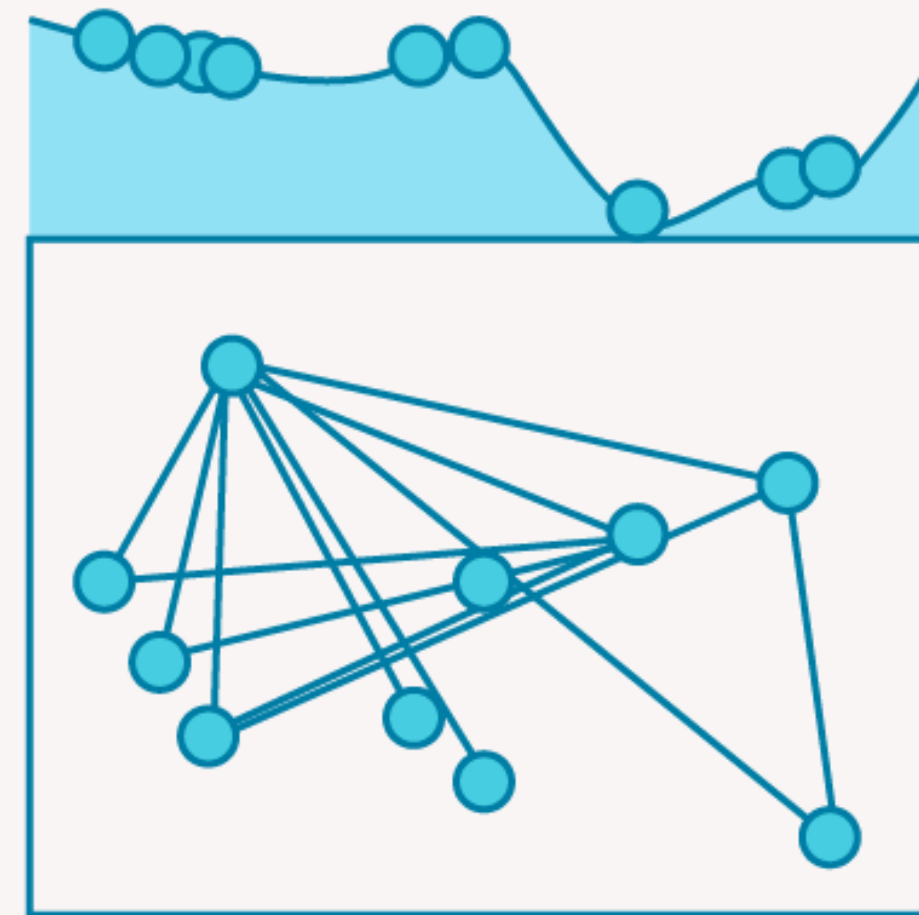
There are $5 \times 4 \times 2 = 40$ possible combinations. A Grid Search would evaluate all 40 combinations.

With Random Search, you specify how many combinations you want to test (e.g., `n_iter = 10`). The algorithm randomly selects 10 different combinations from the 40 possible ones and evaluates only those using cross-validation. The combination with the highest average validation score is chosen.

Grid Search



Random Search




```
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.tree import DecisionTreeClassifier

X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

param_dist = {
    "max_depth": [3, 5, 7],
    "min_samples_split": [2, 4, 6]
}

random = RandomizedSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=5,
    cv=5,
    scoring="accuracy",
    random_state=42
)

random.fit(X_train, y_train)

print(random.best_params_)
print(random.best_score_)
print(random.score(X_test, y_test))
```

Random Search



Bayesian Optimization

Bayesian Optimization is an intelligent hyperparameter tuning method that uses the results of previous trials to decide which hyperparameter combination to try next. Unlike Grid Search, which tests every combination, or Random Search, which samples combinations randomly, Bayesian Optimization learns from each evaluation and focuses on the most promising regions of the search space.



```
import optuna
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier

X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

def objective(trial):
    model = DecisionTreeClassifier(
        max_depth=trial.suggest_int("max_depth", 3, 7),
        min_samples_split=trial.suggest_int("min_samples_split", 2, 6),
        random_state=42
    )

    score = cross_val_score(
        model,
        X_train,
        y_train,
        cv=5,
        scoring="accuracy"
    ).mean()

    return score

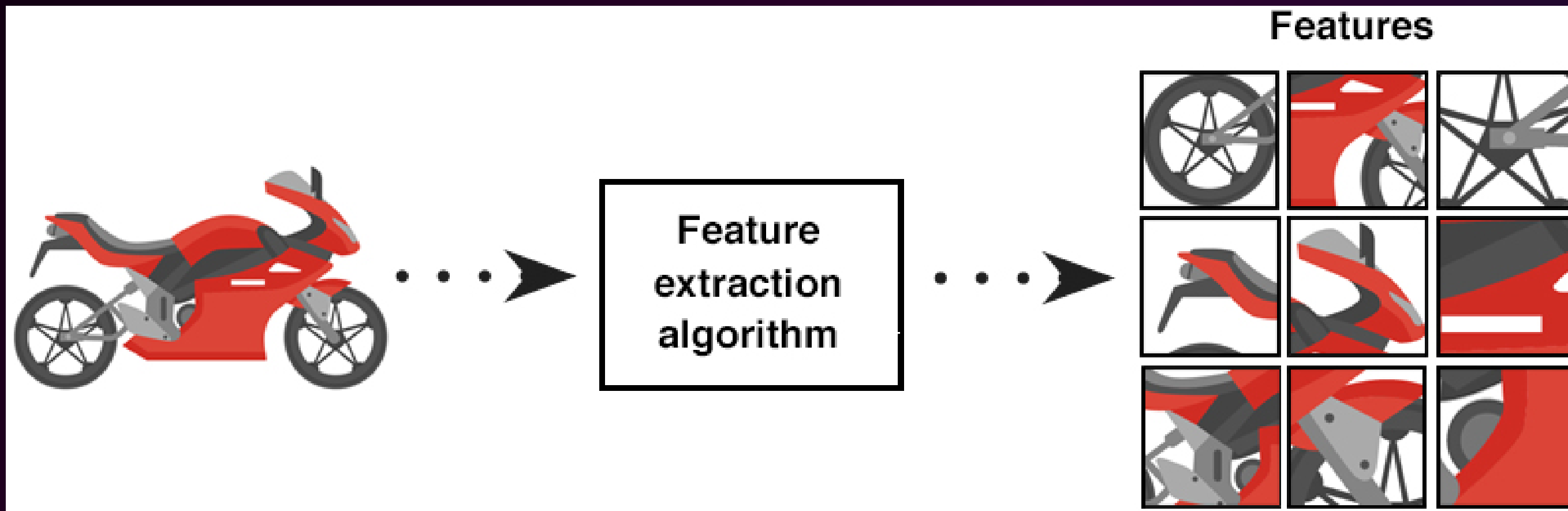
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=20)

best_model = DecisionTreeClassifier(
    **study.best_params,
    random_state=42
)

best_model.fit(X_train, y_train)

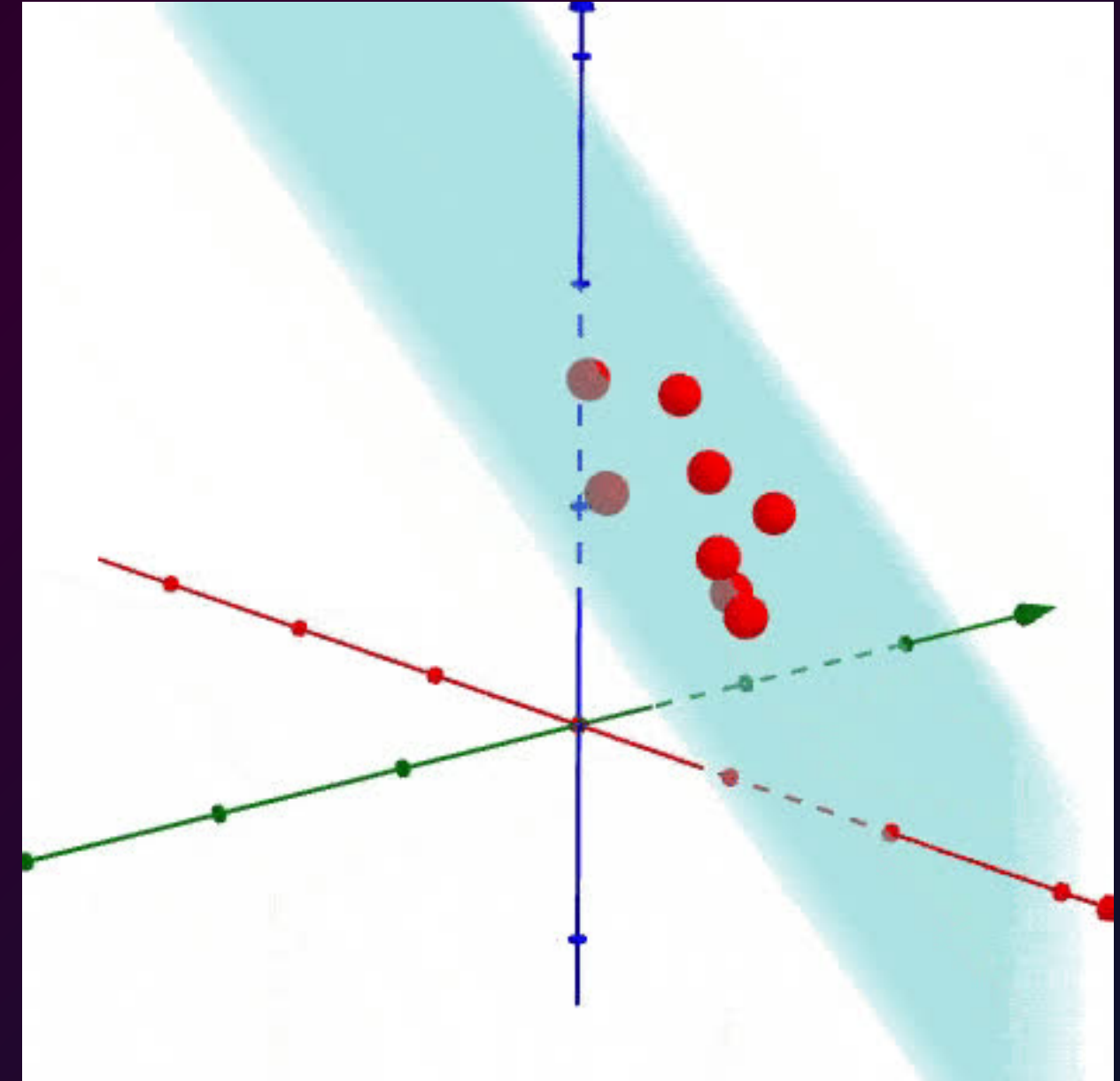
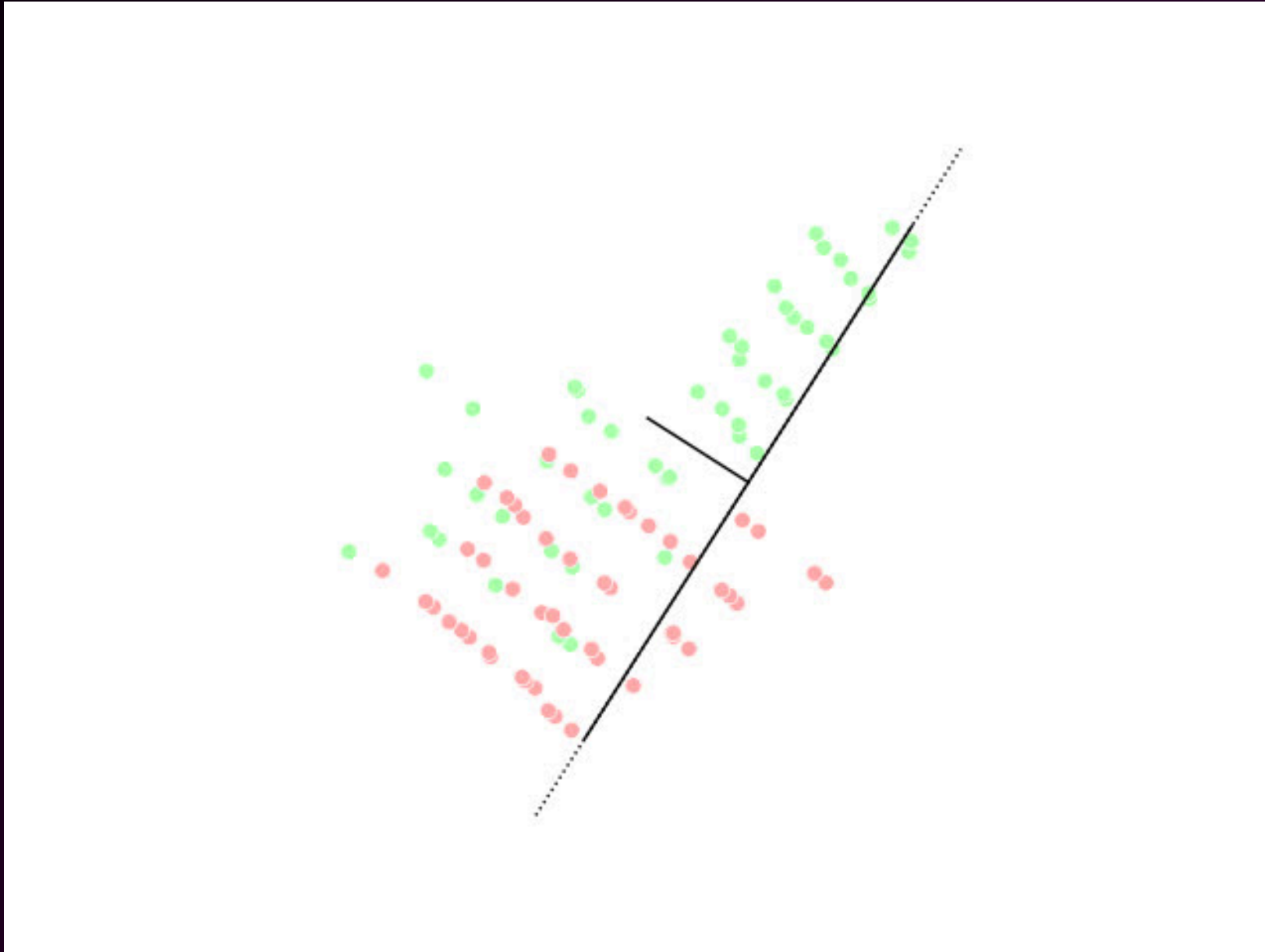
print(study.best_params)
print(study.best_value)
print(best_model.score(X_test, y_test))
```

Features extraction



Feature extraction is the process of transforming the original features into a new set of features that contain the most useful information for a machine learning model. Instead of selecting existing features, feature extraction creates new features by combining or transforming the original ones.

PCA



Principal Component Analysis (PCA) is an unsupervised feature extraction technique used to reduce the number of features in a dataset while preserving as much information (variance) as possible. It transforms the original correlated features into a new set of uncorrelated features called principal components. The first principal component captures the largest amount of variance in the data, the second captures the next largest variance while being orthogonal to the first, and so on. By keeping only the first few principal components, PCA reduces dimensionality, speeds up model training, removes redundancy, and often improves visualization and generalization with minimal information loss.



```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

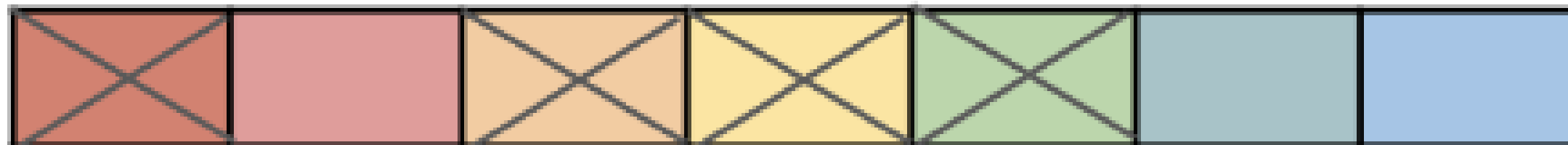
print(X_train.shape)
print(X_train_pca.shape)
```

Features Selection

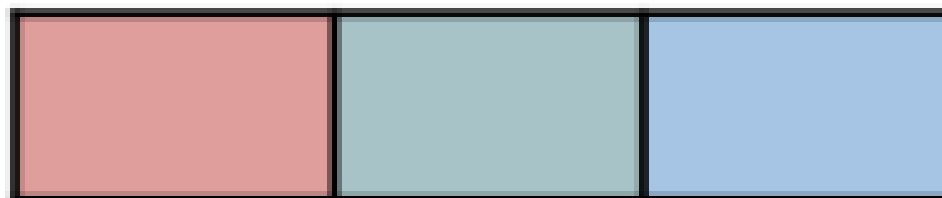
All Features



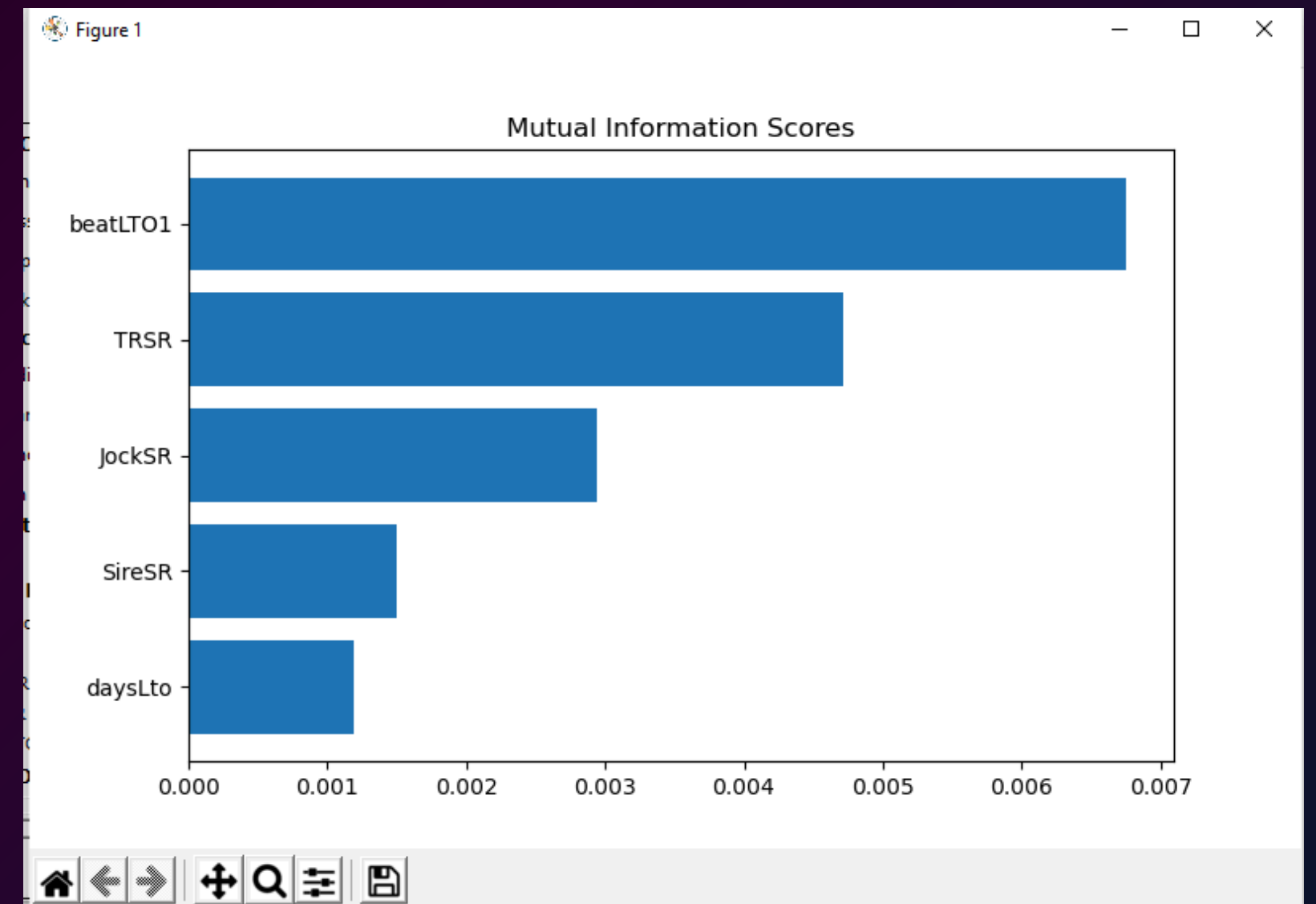
Feature Selection



Final Features



Mutual Information (MI) is a feature selection technique that measures how much information one variable provides about another. In machine learning, it tells us how useful a feature is for predicting the target variable. If knowing the value of a feature greatly reduces the uncertainty about the target, then that feature has high mutual information. If the feature provides no useful information about the target, its mutual information is zero.





```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_selection import mutual_info_classif

mi = mutual_info_classif(X, y, random_state=42)

mi_df = pd.DataFrame({
    "Feature": X.columns,
    "Mutual Information": mi
})

mi_df = mi_df.sort_values("Mutual Information", ascending=True)

plt.figure(figsize=(8, 6))
plt.barh(mi_df["Feature"], mi_df["Mutual Information"])
plt.xlabel("Mutual Information")
plt.ylabel("Features")
plt.title("Feature Importance using Mutual Information")
plt.show()
```

Ensemble Learning

Ensemble Learning

```
graph TD; A[Ensemble Learning] --> B[Bagging]; A --> C[Boosting]; A --> D[Stacking];
```

Bagging

Boosting

Stacking

Bagging

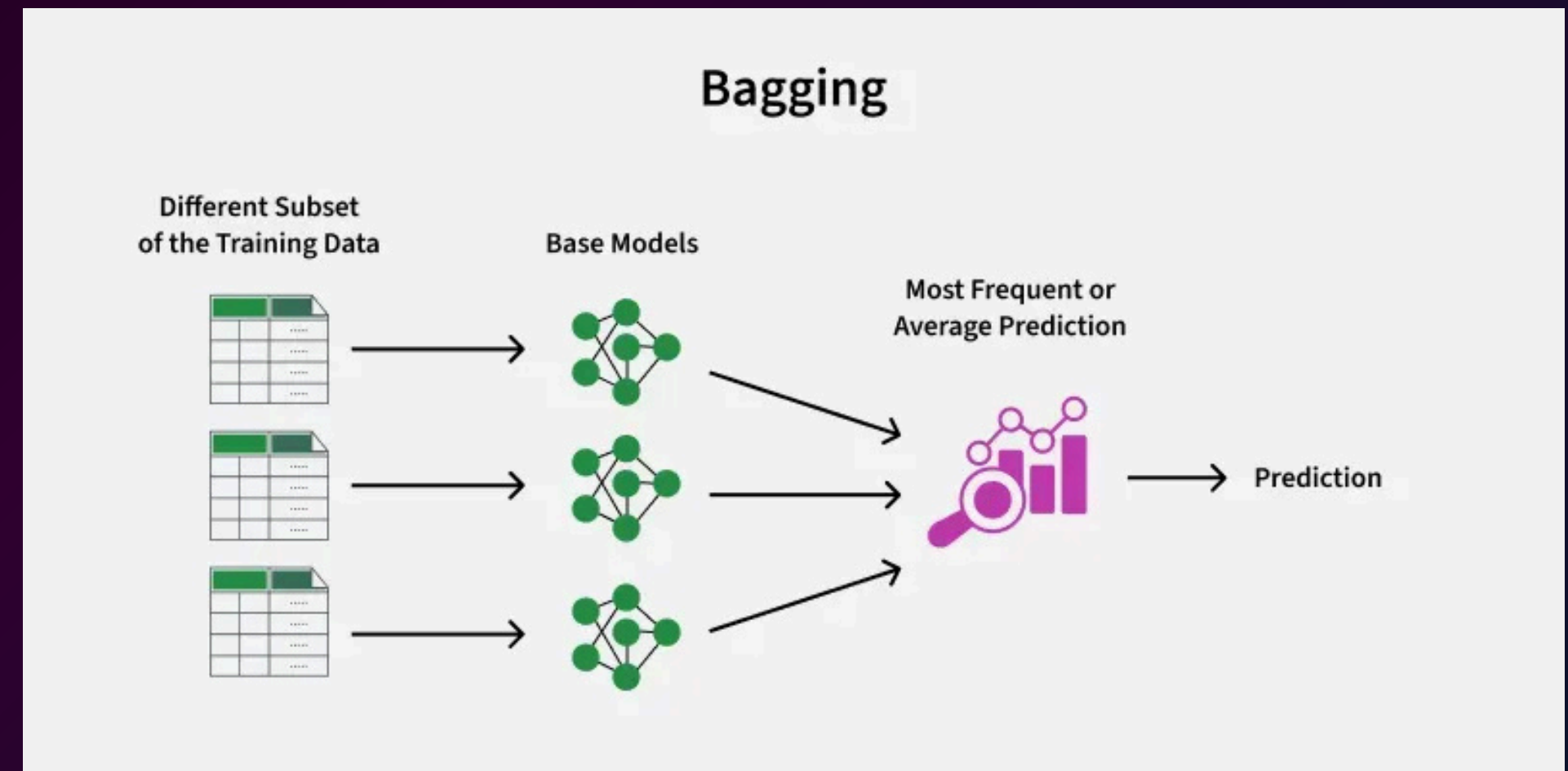
Bagging is a technique that involves creating multiple versions of a model and combining their outputs to improve overall performance.

In bagging several base models are trained on different subsets of the training data, then aggregate their predictions to make the final decision. The subsets of the data are created using bootstrapping, a statistical technique where samples are drawn with replacement, meaning some data points can appear more than once in a subset.

The final prediction from the ensemble is typically made by either:

- Averaging the predictions (for regression problems), or
- Majority voting (for classification problems).

This approach helps to reduce variance, especially with models that are prone to overfitting, such as decision trees.

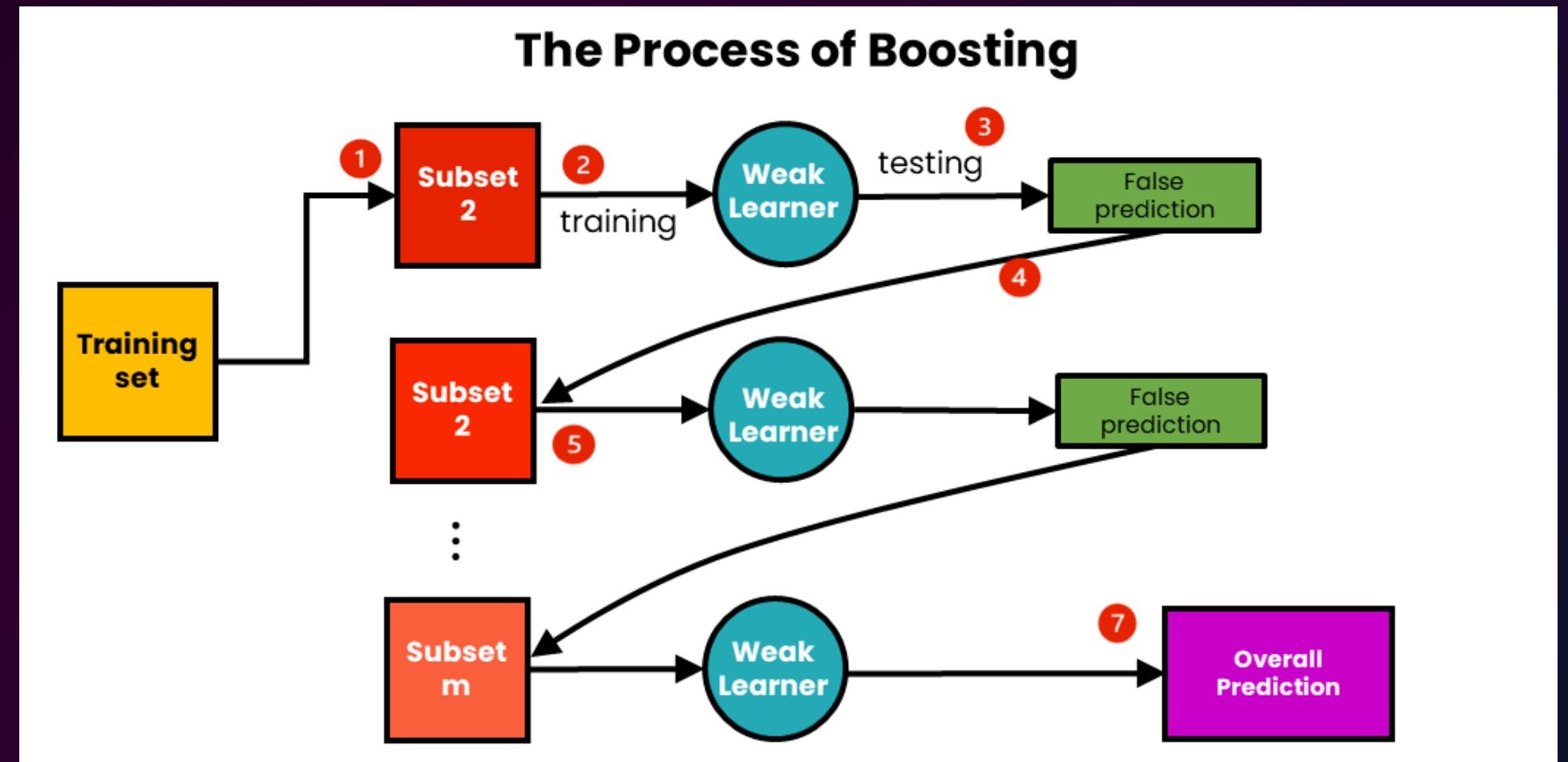


Boosting

Boosting is an ensemble technique where multiple models are trained sequentially, with each new model attempting to correct the errors made by the previous ones.

Boosting focuses on adjusting the weights of incorrectly classified data points, so the next model pays more attention to those difficult cases. By combining the outputs of these models, boosting typically improves the accuracy of the final prediction.

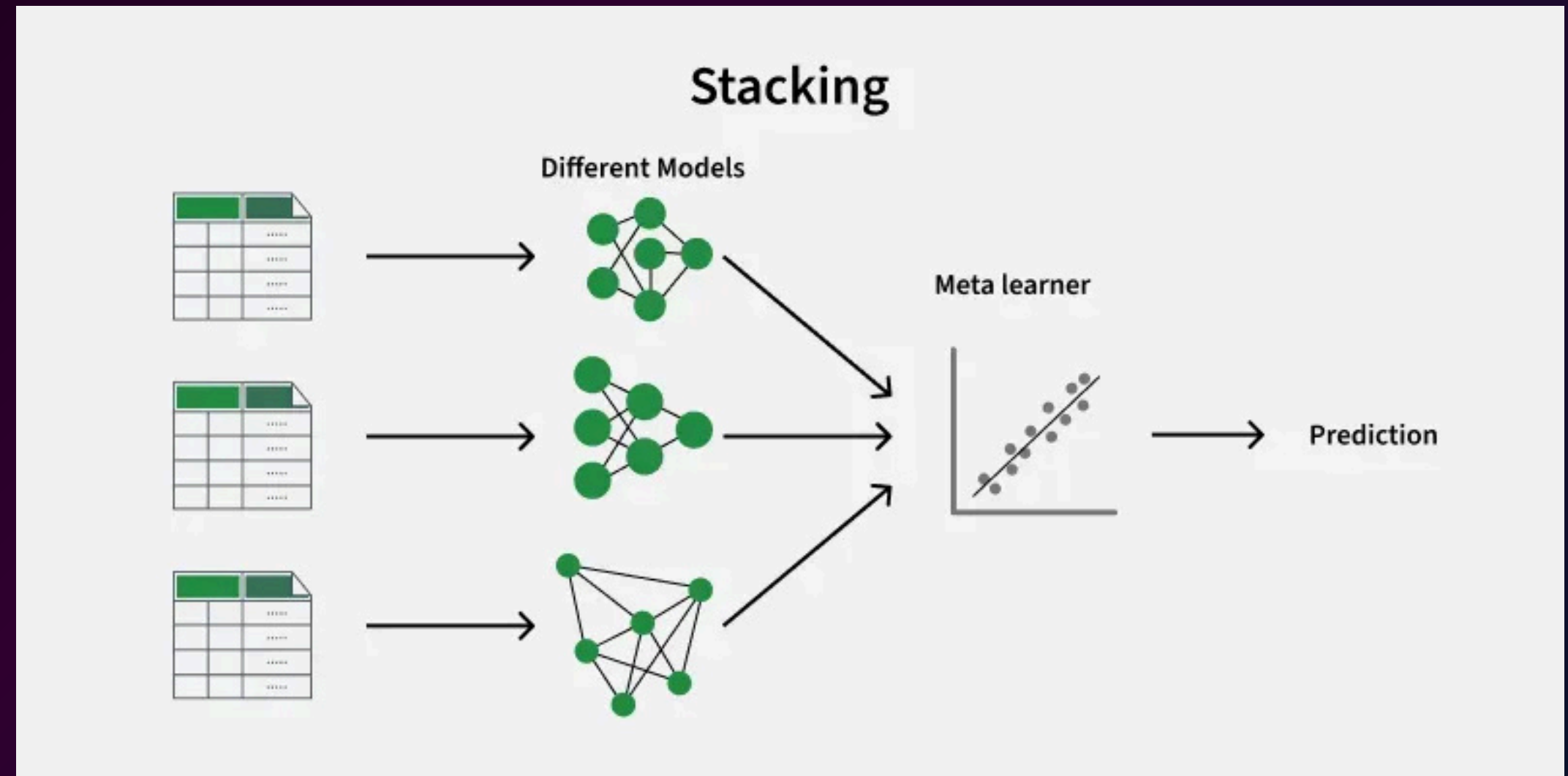
In boosting, each new model is added to the ensemble in a way that emphasizes the mistakes made by previous models. The final prediction is usually made by combining the weighted predictions of all the models in the ensemble.



Stacking

Stacking (Stacked Generalization) combines multiple models (base learners) of different types, where each model makes independent predictions and a meta-model is trained to combine these predictions. Instead of simply averaging or voting, as in bagging and boosting, stacking trains a higher-level model (meta-model) to learn how to best combine the predictions of the base models.

In stacking, the base models are trained on the original data and their predictions are then used as features for the meta-model, which learns how to combine them effectively. The final prediction is made by the meta-model based on the combined outputs of all the base models.



Presented By: Omar Ahmed

Thank You

